Data Structures (ETCCDS202)

Unit 1 **Assignment: Foundations & Algorithmic Analysis**

Course Details

Name of the School: SOET **(School of Engineering & Technology)**

Program: CSE Robotics and AI

Semester: 2

Batch: 2025-2029

Student Name: Ajitesh **Singh Gusain**

1. Introduction

In this assignment, I implemented a Python toolkit called the Algorithmic Efficiency & Recursion Toolkit (AERT).
data structures, Abstract Data Types (ADT), recursion, and algorithmic complexity analysis.

The toolkit includes:

- Stack ADT
- Recursive factorial and Fibonacci
- Tower of Hanoi
- Recursive binary search
- Time and space complexity analysis

Part A: Stack ADT (ADT + Operations)

: ADT and Stack Complexity

- **What is an ADT?** An Abstract Data Type (ADT) is a high-level description of a data structure that defines **what** operations can be performed (like push or pop) but not **how** they are implemented in code.
- **Why is Stack O(1)?** Stack operations like push and pop are usually constant time (O(1)) because they only interact with the "top" element. There is no need to loop through the data or shift other elements, regardless of how large the stack grows.

---

2. Coding Task: Using your StackADT

In this assignment, the Stack ADT is used.

The stack follows the LIFO (Last In, First Out) principle.

Why are stack operations O(1)?

Operations like push, pop, and peek always work on the top element of the stack.
They do not depend on the number of elements present in the stack.
So, their time complexity is O(1).

The assignment requires me to use the StackADT class built in Part A to "track" or "trace" what is happening inside one of my recursive functions.

STACK IMPLEMENTATION

```
class ADT_Stack:
    def __init__(self):
        self.items = []

    def push(self, x):
        self.items.append(x)

    def pop(self):
        if not self.is_empty():
            return self.items.pop()
        return None

    def peek(self):
        return self.items[-1] if not self.is_empty()

    def is_empty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)

    def seek(self):
        return self.items
```

```python
#PART A
# here  i have created an   instance of my stack
searching_stack = ADT_Stack()

def binary_search(arr, key, low, high):
    if low > high:
        return -1

    mid = (low + high) // 2

    # TASK REQUIREMENT: Storing the mid-index in the StackADT
    searching_stack.push(mid)

    if arr[mid] == key:
        return mid
    elif arr[mid] > key:
        return binary_search(arr, key, low, mid - 1)
    else:
        return binary_search(arr, key, mid + 1, high)


# # test case1
# arr = [1, 3, 5, 3, 9, 11, 13,7]
# key=7
# low=0
# high=len(arr)-1

# print(binary_search(arr, key, low, high))
# print(searching_stack.seek())
# # Test case 2
# # arr = [1, 3, 5, 7, 9, 11, 13], searching for
# # key=2


#PART B

def factorial(n):
    # Reject negative inputs gracefully
    if n < 0:
        return "Error: Input must be >= 0"

    # Base Case: 0! = 1
    if n == 0:
        return 1

    # Recursive Step: n * (n-1)!
    return n * factorial(n - 1)
```

```python
#PART C
fibo_counter=0
def fibonacci(n):
    global fibo_counter
    fibo_counter+=1
    if n <=1:
        return n
    else:
        return fibonacci(n-1)+fibonacci(n-2)


dict={}
counter_2=[]
def fib_using_dict(n):
    """Recursive Fibonacci with memoization (Linear Time)"""
    global counter_2
    counter_2+= 1  # Increment every time function is entered

    # Check if value is already in 'memory'
    if n in dict:
        return dict[n]

    # Base case
    if n <= 1:
        return n

    # Store result in memo before returning
    dict[n] = fib_using_dict(n - 1) + fib_using_dict(n - 2)
    return dict[n]


# n=int(input("enter your index for fibonacci sequence:"))
# print(f"value at index {n}:{fibonacci(n)}")


#testing function is testing cases for  all the cases
def tester_function():
    global fibo_counter
    global counter_2



    print("\n--- PART A: Binary Search Test Cases ---")
    arr = [1, 3, 5, 7, 9, 11, 13]
    for key in [7, 1, 13, 2]:
        idx = binary_search(arr, key, 0, len(arr) - 1)
        print(f"Search {key} in {arr}: Found at index {idx}")

    # Edge case: Empty list
    print(f"Search 5 in []: {binary_search([], 5, 0, -1)}")
```

Part B: Factorial & Fibonacci (Recursion + Complexity)

```
--- PART B: Factorial Test Cases ---
Factorial(0) = 1
Factorial(1) = 1
Factorial(5) = 120
Factorial(10) = 3628800
```

The factorial of a number n is defined as

$n! = n \times (n - 1) \times (n - 2) \times \ldots \times 1$

Base case:

$0! = 1$

The recursive function calls itself until it reaches the base case.

Complexity Analysis

- Time Complexity: O(n)
- Space Complexity: O(n) (due to recursion stack)
- Big-O = Big-Θ = O(n)

4. Fibonacci Series

Recursive Fibonacci

In Fibonacci, the function calls itself **two times** for each value.
This causes the same subproblems to be solved again and again.

Why is Fibonacci **slow?**

Because the number of recursive calls increases **exponentially** for larger values of n.

- Time Complexity: **$O(2^n)$**
- Space Complexity: **O(n)**
- **Output**

```
--- PART C: Fibonacci Test Cases ---
n=5 | Result: 5
  recursive fibonacci Calls: 15
n=10 | Result: 55
  recursive fibonacci Calls: 177
n=20 | Result: 6765
  recursive fibonacci Calls: 21891
n=30 | Result: 832040
  recursive fibonacci Calls: 2692537
None
```

<u>Memoized Recursion Logic (Dynamic Programming)</u>

The memoized version of Fibonacci is a top-down dynamic programming approach.

- The Concept: In naive recursion, the function recalculates the same Fibonacci indices multiple times, which is highly inefficient. Memoization solves this by using a "memo" (a dictionary or array) to store the result of each index the first time it is calculated.
- The Process: Before performing any recursive math, the function checks if the value for index n is already in the memo. If it is, the function returns the stored value immediately without further recursion.
- Purpose: This eliminates the "redundant work" seen in the naive version, ensuring that each Fibonacci number is computed exactly once

1. Time Complexity

- Big-O Notation: O(n).
- Explanation: In the naive version, the time complexity is exponential (O(2^n)) because the number of calls doubles with each step. However, with memoization, we only perform a constant amount of work for each value from 0 to n. Since we only compute each number once, the time taken grows linearly with the input size n.
- Proof by Call Counter: As required by your assignment, the call counter will show that for n=30, the naive version takes millions of calls, whereas the memoized version takes approximately 2n calls (roughly 60 calls).

2. Space Complexity

- Big-O Notation: O(n).
- Explanation: The space complexity is determined by two factors:
    1. Recursion Stack: The maximum depth of the call stack is n (the function calls n, n-1, n-2 \dots 0), which takes O(n) space.

2. Memoization Table: We store n results in a dictionary or array, which also takes O(n) space.

- Total Space: O(n) + O(n) = O(n).

3. Best/Average/Worst Cases

- Worst Case: O(n)—when we calculate the sequence for the first time.
- Best Case: O(1)—if the value is already in the memo (e.g., if you call fib_memo(30) twice, the second time it returns the answer instantly).

Summary for Report

Feature	Naive Fibonacci	Memoized Fibonacci
Paradigm	Pure Recursion	Dynamic Programming
Time Complexity	O(2^n) (Exponential)	O(n) (Linear)
Space Complexity	O(n) (Stack depth)	O(n) (Stack + Memo)
Efficiency	Extremely slow for large n	Very fast for large n

```
--- PART D: Fibonacci using meiosiation Test Cases ---
n=5 | Result: 5
  recursive fibonacci Calls: 9
n=10 | Result: 55
  recursive fibonacci Calls: 11
n=20 | Result: 6765
  recursive fibonacci Calls: 21
n=30 | Result: 832040
  recursive fibonacci Calls: 21
None
```

Part C: Tower of Hanoi (Recursion + Trace)

3. Tower of Hanoi and Recursion

The Tower of Hanoi is a classic recursive puzzle that demonstrates how complex tasks can be reduced to simpler versions of themselves.

- **The Subproblem Idea:** To move n disks from Source to Destination, you must first move the top n-1 disks to an "Auxiliary" peg. Once the largest disk is moved to the destination, you then move the n-1 disks from the auxiliary peg to the destination.
- **The Base Case:** Recursion cannot continue infinitely. The baare pegged,arepeg,peg and the function returns.

1. Implementation

This function follows the standard recursive logic: move (n-1) disks to the auxiliary peg, move the (n^{th}) disk to the destination, and finally move the (n-1) disks from the auxiliary to the destination.

2. Required Explanations for your Report

- **Why the number of moves is (2^n – 1):**

To move a stack of n disks, you must:

1. Move n-1 disks to the auxiliary peg.
2. Move the largest disk to the destination.
3. Move the n-1 disks from the auxiliary to the destination.

If M(n) is the number of moves, the relationship is M(n) = 2M(n-1) + 1.

M(2)=3, and

- **Time Complexity O(2^n):**

Since the algorithm makes two recursive calls for every disk (except the base case), the number of operations doubles with each additional disk. Specifically, the recurrence T(n) = 2T(n-1) + 1 results in O(2^n).

- **Space Complexity O(n):**

The space complexity is determined by the maximum depth of the recursion stack. In this algorithm, the depth of the recursion tree is exactly n, as we only reduce n by 1 in each step until we reach the base case.

```
--- PART C (Hanoi) ---
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
None
```

Part D: Recursive Binary Search (Divide & Conquer + Recurrence)

5. Recursive Binary Search

Binary search works by dividing the array into halves.

Steps:

1. Find middle element
2. Compare with key
3. Search left or right half recursively

This follows the divide and conquer paradigm.

Recurrence Relation

$T(n) = T(n/2) + O(1)$

Complexity

- Best Case: $O(1)$
- Average Case: $O(\log n)$
- Worst Case: $O(\log n)$

Output

```
--- PART A: Binary Search Test Cases ---
Search 7 in [1, 3, 5, 7, 9, 11, 13]: Found at index 3
Search 1 in [1, 3, 5, 7, 9, 11, 13]: Found at index 0
Search 13 in [1, 3, 5, 7, 9, 11, 13]: Found at index 6
Search 2 in [1, 3, 5, 7, 9, 11, 13]: Found at index -1
Search 5 in []: -1
```

Option A: Fibonacci Trace

As your Fibonacci function runs, you push the current value of n onto your stack every time the function is called. At the end, I can print the stack to show the "history" of the function calls.

Option B: Tower of Hanoi Moves

Instead of just printing "Move disk 1 from A to C," you push that string into your stack. After the recursion finished, you can pop them out or iterate through the stack to print the final sequence of moves.

Option C: Binary Search Mid-Indices

Every time your binary_search calculates a mid index, you push that index into your stack. He targets.

Which one should you choose?

I can choose **Option B (Tower of Hanoi)** or **Option C (Binary Search)**. They are often the easiest to implement because you are simply storing a result you already calculated in your code.

I chose binary search for my project.

Part E: Paradigm Reflection

1. Binary Search as Divide & Conquer

Binary search is a quintessential example of the divide **&** conquer paradigm because it solves a problem by breaking it down into smaller, more manageable subproblems.

- **Divide:** The algorithm looks at the middle element of a sorted array and compares it to the target. It then "divides" the search space in half.
- **Conquer:** Instead of searching the entire array, it recursively (or iteratively) searches only the half that could contain the target.
- **Why it fits:** It fits this paradigm because the original problem (searching size n) is reduced to an identical problem of a smaller size (n/2) until a base case is reached—either the element is found, or the range is empty.

2. <u>Memoized Fibonacci as Dynamic Programming</u>

While the standard recursive Fibonacci algorithm is highly inefficient, adding **memoization** transforms it into a **Dynamic Programming (DP)** solution.

- **The Problem Solved:** Memoization solves the issue of **overlapping subproblems**. In a naive recursion, Fib(5) calls Fib(4) and Fib(3). However, Fib(4) *also* calls Fib(3). Without memoization, the computer recalculates Fib(3) (and everything below it) multiple times.
- **The Solution:** Memoization stores the result of each subproblem in a table (like an array or dictionary). When the algorithm needs a value it has already computed, it simply "looks it up" in O(1) time rather than re-executing the entire recursive branch. This reduces the time complexity from exponential O(2^n) to linear O(n).

<u>7. Conclusion</u>

This assignment helped me understand:

- Difference between ADT and Data Structure
- Working of recursion and call stack

- Importance of time and space complexity
- Algorithmic paradigms like Divide & Conquer and Dynamic Programming